

Лекция 13: многопоточность

Функциональное программирование на Haskell

Алексей Романов

4 сентября 2026 г.

МИЭТ

Два вида многопоточности: конкурентность и параллелизм

- Все программы, которые мы писали до сих пор, были однопоточными.
- У современных компьютеров много ядер, мы задействуем только одно.
- Чтобы использовать их, нужна многопоточность.

Два вида многопоточности: конкурентность и параллелизм

- Все программы, которые мы писали до сих пор, были однопоточными.
- У современных компьютеров много ядер, мы задействуем только одно.
- Чтобы использовать их, нужна многопоточность.
- Параллелизм: физическая многопоточность. Много ядер (или других ресурсов) используются для ускорения выполнения одной задачи.

Два вида многопоточности: конкурентность и параллелизм

- Все программы, которые мы писали до сих пор, были однопоточными.
- У современных компьютеров много ядер, мы задействуем только одно.
- Чтобы использовать их, нужна многопоточность.
- Параллелизм: физическая многопоточность. Много ядер (или других ресурсов) используются для ускорения выполнения одной задачи.
- Конкурентность: логическая многопоточность. Несколько задач, которые могут выполняться логически одновременно. А значит, потенциально и физически одновременно.

Детерминированность

- Модель программирования детерминирована, если результат программы зависит только от полученных данных (аргументов и ввода пользователя).

Детерминированность

- Модель программирования детерминирована, если результат программы зависит только от полученных данных (аргументов и ввода пользователя).
- Недетерминирована, если результат зависит от того, в каком порядке произойдут события (или от сгенерированных случайных чисел и т.д.).

Детерминированность

- Модель программирования детерминирована, если результат программы зависит только от полученных данных (аргументов и ввода пользователя).
- Недетерминирована, если результат зависит от того, в каком порядке произойдут события (или от сгенерированных случайных чисел и т.д.).
- Обычно параллельные программы детерминированы, конкурентные нет.
- Недетерминированность усложняет тестирование, отладку и понимание программ, но часто без неё не обойтись.
- Тестирование свойств это пример пользы от недетерминированности.

Параллелизм: монада <MINTED>

- Модуль <MINTED> в пакете `parallel`.
- Тип <MINTED> — «строгая» монада: в <MINTED> действие `mx` выполняется до вызова `f`. Но <MINTED> не вычисляет `x`, а лишь создаёт «искру» (spark).

Параллелизм: монада <MINTED>

- Модуль <MINTED> в пакете `parallel`.
- Тип <MINTED> — «строгая» монада: в <MINTED> действие `mx` выполняется до вызова `f`. Но <MINTED> не вычисляет `x`, а лишь создаёт «искру» (spark).
- Есть функции <MINTED>
- <MINTED>: запустить вычисление `x` и сразу вернуться.
- <MINTED>: вернуться после вычисления `x`.
- <MINTED>: запустить вычисление, описанное в `x`, и сразу вернуться.

- Запуск двух вычислений параллельно:
<MINTED>

Примеры <MINTED>

- Запуск двух вычислений параллельно:
<MINTED>
- Запуск двух вычислений параллельно и возврат,
когда оба закончатся:
<MINTED>

- <MINTED>

Фибоначчи в <MINTED>

- <MINTED>
- <MINTED>

Фибоначчи в <MINTED>

- <MINTED>
- <MINTED>
- Здесь для простоты кода много повторных вычислений!

Чтобы это работало

- По умолчанию GHC собирает программу с однопоточной средой выполнения, и все **<MINTED>** (и **<MINTED>** далее) работают на одном ядре.
- Нужны опция компиляции **<MINTED>** и опция среды выполнения **<MINTED>** (все ядра) или **<MINTED>**:
<MINTED>
(в **<MINTED>** лабораторных это **<MINTED>**).

- **<MINTED>** печатает статистику искр: **<MINTED>**.
Converted — выполнены параллельно, *fizzled* —
главный поток успел вычислить значение сам,
GC'd — результат никому не понадобился.

Искры и их цена

- **<MINTED>** печатает статистику искр: **<MINTED>**.
Converted — выполнены параллельно, *fizzled* — главный поток успел вычислить значение сам, *GC'd* — результат никому не понадобился.
- Искра дешевле потока ОС, но не бесплатна.
<MINTED> без порога делает $O(\varphi^n)$ искр на $O(1)$ работы каждая и работает *медленнее* последовательного варианта. Обычно: ниже некоторого n переходить на обычную рекурсию, а список делить **<MINTED>**.

Стратегии вычислений

- **<MINTED>**.
- **<MINTED>** и **<MINTED>** — стратегии.
- **<MINTED>** — стратегия, которая ничего не вычисляет.
- Стратегия принимает на вход thunk, вычисляет его части с помощью **<MINTED>**, **<MINTED>** и **<MINTED>**.
- Например, обобщая пример с прошлого слайда **<MINTED>**

Стратегии вычислений

- **<MINTED>**.
- **<MINTED>** и **<MINTED>** — стратегии.
- **<MINTED>** — стратегия, которая ничего не вычисляет.
- Стратегия принимает на вход thunk, вычисляет его части с помощью **<MINTED>**, **<MINTED>** и **<MINTED>**.
- Например, обобщая пример с прошлого слайда **<MINTED>**
(или **<MINTED>**)
- **<MINTED>**

- Обобщим **<MINTED>** дальше, чтобы она принимала стратегии на вход:

<MINTED>

Комбинаторы стратегий

- Обобщим **<MINTED>** дальше, чтобы она принимала стратегии на вход:
<MINTED>
- **<MINTED>** запускает стратегию параллельно.
- Например, **<MINTED>** и **<MINTED>** эквивалентны **<MINTED>**.

Комбинаторы стратегий

- Обобщим **<MINTED>** дальше, чтобы она принимала стратегии на вход:
<MINTED>
- **<MINTED>** запускает стратегию параллельно.
- Например, **<MINTED>** и **<MINTED>** эквивалентны **<MINTED>**.
- **<MINTED>**

Комбинаторы стратегий для списков

- С помощью стратегий достаточно легко сделать параллельные `map/filter`/и т.д.
- Например, используя **<MINTED>**:
<MINTED>

Комбинаторы стратегий для списков

- С помощью стратегий достаточно легко сделать параллельные `map/filter`/и т.д.
- Например, используя **<MINTED>**:
<MINTED>
- Это скорее всего создаст слишком много «искр», лучше использовать
 - **<MINTED>**: разбивает список на части одинаковой длины и работает параллельно с каждой частью
 - **<MINTED>**: начинает работать параллельно с первыми элементами списка, добавляет следующие при использовании результатов

Монада **<MINTED>**: параллелизм без ленивости

- Модуль **<MINTED>** в пакете **<MINTED>**.
- Тип **<MINTED>** — тоже монада, описывающая процесс вычисления значения типа **a** с параллельными частями.
- **<MINTED>** создаёт новый поток.
- **<MINTED>** производит вычисление и возвращает его результат.

Монада <MINTED>: параллелизм без ленивости

- Модуль <MINTED> в пакете <MINTED>.
- Тип <MINTED> — тоже монада, описывающая процесс вычисления значения типа `a` с параллельными частями.
- <MINTED> создаёт новый поток.
- <MINTED> производит вычисление и возвращает его результат.
- Потоки общаются между собой с помощью <MINTED>, которые можно записать только 1 раз.
- В результате не должно быть <MINTED> или ссылок на них!
- При соблюдении этого условия вычисления в <MINTED> детерминированы.

- Интерфейс <MINTED>:
<MINTED>
- Значения, положенные в <MINTED>, полностью вычисляются (есть ещё <MINTED> и <MINTED>).
- Переменной можно дать значение только 1 раз, иначе исключение.
- При вызове `get` для переменной, ещё не имеющей значения, начинается ожидание.

Примеры <MINTED>

- <MINTED>: аналог <MINTED>, но возвращающий переменную, в которой будет сохранено вычисленное значение

<MINTED>

- Параллельный `map`:

<MINTED>

Как и в прошлый раз, создаёт «поток» для каждого элемента, для простоты кода.

<MINTED>

- Перейдём от параллелизма к конкурентности.
- Модуль **<MINTED>** в пакете **<MINTED>** и **<MINTED>/<MINTED>/<MINTED>**.
- **<MINTED>** — ссылка на поток.
- **<MINTED>** создаёт новый поток, который произведёт данные действия и выйдет, возвращает ссылку на созданный поток.

Конкурентность

- Перейдём от параллелизма к конкурентности.
- Модуль **<MINTED>** в пакете **<MINTED>** и **<MINTED>/<MINTED>/<MINTED>**.
- **<MINTED>** — ссылка на поток.
- **<MINTED>** создаёт новый поток, который произведёт данные действия и выйдет, возвращает ссылку на созданный поток.
- Это так называемый «зелёный поток», то есть не поток операционной системы, а похожий по поведению объект виртуальной машины Haskell.

<MINTED>: подводные камни

- Когда `main` завершается, программа завершается целиком: остальные потоки просто убиваются. Дождаться их — ваша забота (`MVar` на следующем слайде или `async`).
- Исключение в потоке `<MINTED>` печатается в `<MINTED>` и теряется; в `main` оно не попадает.
- Как и для `<MINTED>`, второе ядро появится только с `<MINTED>` и `<MINTED>`; без них зелёные потоки всё равно работают, но по очереди на одном ядре.

- Потоки могут взаимодействовать через **MVar**, которые очень похожи на **<MINTED>**, но записанное значение можно убрать или изменить.
- Переменная в любой момент или пустая или полная (содержит значение).
<MINTED>
- Если **<MINTED>** вызвана на полной переменной, она ждёт, пока та не опустеет.
- **<MINTED>** и **<MINTED>** отличаются тем, что первая убирает прочитанное значение. На пустой переменной обе ждут.

- Реализация вычисления чисел Фибоначчи с помощью **MVar** не сильно отличается от **<MINTED>** (только аналога **<MINTED>** в стандартной библиотеке нет, смотрите пакет **<MINTED>**):
<MINTED>

Правильный Фибоначчи

- Уже упоминалось, что приведённые программы для вычисления чисел Фибоначчи имеют серьёзный недостаток.
- Например, **<MINTED>** вызовет параллельно **<MINTED>** и **<MINTED>**, а **<MINTED>** снова вызовет **<MINTED>**.
- Чтобы этого избежать, нужно добавить вспомогательный аргумент для хранения уже полученных результатов.
- Его тип может быть:
 - Для **<MINTED>**: **<MINTED>**.
 - Для **<MINTED>**: **<MINTED>**.
 - Для **IO**: **<MINTED>**, хотя **<MINTED>** тоже подойдёт.

- **MVar** можно рассматривать как каналы, хранящие не более одного значения.
- **<MINTED>** это канал для произвольного количества значений.
<MINTED>
- **<MINTED>** сохраняет значение в конец очереди.
- **<MINTED>** читает из её начала и ждёт, если она пуста.

- На практике вместо голых `<MINTED>` и `MVar` используют `<MINTED>`:
`<MINTED>`
- `<MINTED>` перебрасывает исключение из потока в вызывающий, а `<MINTED>/<MINTED>` убивают дочерний поток, если родитель завершился (в том числе исключением) — потоки не «утекают».
- Фибоначчи: `<MINTED>`.

- Понятие транзакций может быть знакомо по базам данных.
- Это наборы операций, которые все вместе либо выполняются успешно, либо откатываются.
- В Haskell это позволяет делать монада **STM** (пакет **stm**) с операциями над **<MINTED>**:
<MINTED>

Транзакции (2)

- **<MINTED>** выполняет транзакцию целиком: другие потоки не видят промежуточных состояний, а при конфликте транзакция повторяется.
- Типы гарантируют, что у действия в монаде **STM** не может быть побочных эффектов, кроме изменения значений **<MINTED>**.
- Обычно проще написать правильный код, который работает с несколькими **<MINTED>**, чем с **MVar**, так как можно не беспокоиться о видимости промежуточных состояний.

Дополнительное чтение

- [Parallel and Concurrent Programming in Haskell](#) (книга, 2013, доступна бесплатно)
- [A Tutorial on Parallel and Concurrent Programming in Haskell](#)
- [Erlang](#) Функциональный язык, построенный на модели акторов (взаимодействующих только через отправку сообщений друг другу вместо общих переменных).
- [Ответ на Stack Overflow про разницу между <MINTED> и <MINTED>](#) (ещё один в конце главы 4 первой ссылки).
- [Streamly: Beautiful Streaming, Concurrent and Reactive Composition](#)
- [Ivish](#) Пакет, обобщающий <MINTED> на основе теории решёток. К сожалению, мёртв с 2014.